Studio Nx

JS 106 — secondsToTicks()

2026-10-14

JS 106 — `secondsToTicks()` (초를 tick으로 되돌리기)

항목	내용
파일	`studionx/frontend/static/js/audio/timing.js` (함수 추가, 소리와 화면을 쓰지 않는 계산 전용 모듈)
작업	S1-05 오디오 엔진 기초: 세 번째 함수 (FR-202a 곡 중간 템포 변경의 역변환, FR-205 시간 단위 tick, PPQ 960)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	스케줄러(재생 중 위치를 tick 으로 잡고 다음 소리 시각을 예약), 메트로놈, 루프 순서로 한 함수씩 진행합니다

1. 하는 일

`ticksToSecondsWithChanges` (105) 의 역함수입니다. 초(재생 위치)를 tick으로 되돌립니다. 재생 중인 위치를 화면에 표시하거나, 루프 끝을 tick으로 잡을 때 씁니다.

입력	결과
0.5초, 120 BPM, 변경 없음	960 tick (1박)
2초, 120 BPM, 변경 없음	3840 tick (한 마디)
120→60 STEP 곡에서 3초	3840 + 960 = 4800 tick
120→240 LINEAR 곡에서 3840 tick의 초	3840 tick (왕복 확인)
−0.5초 (카운트인)	−960 tick

`ticksToSecondsWithChanges(secondsToTicks(초))` 가 원래 초로, `secondsToTicks(ticksToSecondsWithChanges(tick))` 가 원래 tick으로 돌아옵니다(왕복 시험).

2. 코드

```
export function secondsToTicks(seconds, baseBpm, changes) {
  if (typeof seconds !== 'number' || !isFinite(seconds)) throw new RangeError('seconds 는 유한한 숫자여야 합니다: ' + seconds);
  if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm 은 0 보다 큰 숫자여야 합니다: ' + baseBpm);

  const list = [];
  if (changes != null) {
    if (!Array.isArray(changes)) throw new RangeError('changes 는 배열이어야 합니다: ' + changes);
    for (let i = 0; i < changes.length; i++) {
      const c = changes[i];
      if (c == null) continue;
      if (typeof c.tick !== 'number' || !isFinite(c.tick) || c.tick <= 0) throw new RangeError('변경점 tick 은 0 보다 큰 유한한 숫자여야 합니다: ' + c.tick);
      if (typeof c.bpm !== 'number' || !isFinite(c.bpm) || c.bpm <= 0) throw new RangeError('변경점 bpm 은 0 보다 큰 숫자여야 합니다: ' + c.bpm);
      list.push({ tick: c.tick, bpm: c.bpm, ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' });
    }
    list.sort(function (a, b) { return a.tick - b.tick; });
  }
```

```
  if (seconds <= 0) return (seconds * baseBpm * PPQ) / 60;

  let rest = seconds;
  let prevTick = 0;
  let bpm = baseBpm;
  for (let i = 0; i < list.length; i++) {
    const c = list[i];
    if (c.tick <= prevTick) continue;
    const span = c.tick - prevTick;
    const isLinear = c.ramp === 'LINEAR' && c.bpm !== bpm;
    const slope = isLinear ? (c.bpm - bpm) / span : 0;
    const segSeconds = isLinear
      ? (60 / PPQ) * (1 / slope) * Math.log(c.bpm / bpm)
      : (span * 60) / (bpm * PPQ);

    if (rest >= segSeconds) {
      rest -= segSeconds;
      prevTick = c.tick;
      bpm = c.bpm;
      continue;
    }
    if (isLinear) {
      const bpmAt = bpm * Math.exp((rest * PPQ * slope) / 60);
      return prevTick + (bpmAt - bpm) / slope;
    }
    return prevTick + (rest * bpm * PPQ) / 60;
  }
  return prevTick + (rest * bpm * PPQ) / 60;
}
```

최신 문법으로 쓰면 (공부용)

```
export const secondsToTicks = (seconds, baseBpm, changes) => {
  let rest = seconds;
  let prevTick = 0;
  let bpm = baseBpm;
  for (const c of (changes ?? [])) {
    // …구간 계산은 같음
  }
  return prevTick + (rest * bpm * PPQ) / 60;
};
```

최신 문법	실제 코드	뜻
`for (const c of arr)`	`for (let i = 0; …)`	배열의 값을 차례로 씁니다
`changes ?? []`	`if (changes != null) …`	값이 없으면 빈 배열로 봅니다
`Math.exp`	(같음)	실제 코드도 `Math.exp` 를 씁니다. `ln` 의 역으로 지수(e의 거듭제곱)를 구합니다

3. 왜 이렇게

선택	이유
구간을 차례로 지나가며 "남은 초"를 줄여 나감	105와 같은 방식이라 두 함수의 결과가 서로 맞습니다. 변경점이 몇 개뿐이라 충분히 빠릅니다
LINEAR 은 지수(`Math.exp`)로 풂	105에서 초 = 60/PPQ × (1/slope) × ln(bpm(t)/bpmStart) 였으므로, 이를 bpm(t) 에 대해 풀면 지수가 됩니다. `bpm(t)` 을 구한 뒤 `t = prevTick + (bpm(t)-bpmStart)/slope` 로 위치를 냅니다
`rest >= segSeconds` 로 구간 경계 처리	남은 초가 구간 길이와 정확히 같으면 다음 구간으로 넘겨 그 변경점 위치를 돌려줍니다. 왕복 시 경계에서 값이 어긋나지 않습니다
초가 음수면 시작 템포로	카운트인(FR-203)은 0보다 앞이라 템포 변경이 없습니다. 105의 음수 처리와 역이 맞습니다
소수점 tick 을 그대로 돌려줌	반올림(마디·박 표시)은 부르는 쪽이 정합니다. 여기서 미리 반올림하면 왕복 값이 틀어집니다
오류 검사는 105와 같게	같은 입력 규칙(tick 은 0보다 큼, bpm 은 0보다 큼)을 씁니다

4. 시험 (Node에서 직접 실행, 37개 통과 · 105의 23개도 다시 통과)

확인	결과
기본 값(0.5초=960, 2초=3840, 0초=0, −0.5초=−960)	✓
왕복: tick→초→tick 18가지(변경 없음, STEP 1·2개, LINEAR 오르막·내리막, 섞임)	✓
왕복: 초→tick→초 6가지	✓
변경점 경계에서 정확히 그 tick	✓
정렬하지 않은 입력	✓
seconds · baseBpm · changes · 변경점 값 오류 시 `RangeError`	✓